

P.Murali Krishna
(192525178)

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5

5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5
8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I P.Murali Krishna (192525178) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques	Uses appropriate debugging	Limited debugging approach;	Unable to debug or resolve issues

		effectively to resolve issues	methods with minor errors	requires guidance	
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Answers

Q1. Fix Incorrect JOIN Syntax & Duplicate Results (5 Marks)

Buggy Query:

```
SELECT E.Name, D.DeptName
FROM Employee E, Department D
WHERE E.Salary > 50000;
-- Missing join condition → Cartesian product → duplicates
```

Fixed Query:

```
SELECT DISTINCT E.Name, D.DeptName
FROM Employee E
INNER JOIN Department D ON E.DeptID = D.DeptID
WHERE E.Salary > 50000;
```

Error: Missing join condition caused Cartesian product. Solution: Explicit INNER JOIN with ON clause; DISTINCT if needed.

Q2. Debug Sub-query with Missing Correlation (5 Marks)

Buggy Query:

```
SELECT Name, Salary FROM Employee
```

```
WHERE Salary > (SELECT AVG(Salary) FROM Employee);  
-- Compares with overall average, not department average
```

Corrected (Correlated) Query:

```
SELECT E1.Name, E1.Salary, E1.DeptID  
FROM Employee E1  
WHERE E1.Salary > (  
    SELECT AVG(E2.Salary)  
    FROM Employee E2  
    WHERE E2.DeptID = E1.DeptID  
);
```

Error: Missing correlation predicate. Inner query must reference outer DeptID.

Q3. Fix SQL View Definition Errors (5 Marks)

Buggy View:

```
CREATE VIEW DeptSummary AS  
SELECT DeptID, COUNT(*), AVG(Salary)  
FROM Employee GROUP BY DeptID;  
-- Aggregates without aliases; may fail in some DBMS
```

Corrected & Optimized View:

```
CREATE VIEW DeptSummary AS  
SELECT E.DeptID, D.DeptName,  
       COUNT(E.EmpID) AS EmpCount,  
       ROUND(AVG(E.Salary), 2) AS AvgSalary  
FROM Employee E  
JOIN Department D ON E.DeptID = D.DeptID  
GROUP BY E.DeptID, D.DeptName;
```

Fixed: Added aliases, joined for DeptName, used ROUND.

Q4. Debug Relational Algebra – Incorrect Natural Join (5 Marks)

Buggy Expression:

```
 $\pi$  Name, DeptName (Employee  $\times$  Department)  
-- Cartesian product, not natural join
```

Correct Expression:

```
 $\pi$  Name, DeptName (Employee  $\bowtie$  Department)  
-- or:  
 $\pi$  Name, DeptName ( $\sigma$  Employee.DeptID = Department.DeptID (Employee  $\times$  Department))
```

Error: Used $\times$ instead of $\bowtie$. Natural join matches on common attributes (DeptID).

Q5. Fix Referential Integrity Violation (5 Marks)

Violating Statement:

```
INSERT INTO Employee (EmpID, Name, Salary, DeptID)
VALUES ('E200', 'Test User', 45000, 'D99');
-- D99 does not exist → FK violation
```

Correct Approach:

```
INSERT INTO Department (DeptID, DeptName, Location)
VALUES ('D99', 'Research', 'Pune');

INSERT INTO Employee (EmpID, Name, Salary, DeptID, ManagerID)
VALUES ('E200', 'Test User', 45000, 'D99', NULL);

ALTER TABLE Employee
ADD CONSTRAINT fk_emp_dept FOREIGN KEY (DeptID)
REFERENCES Department(DeptID)
ON DELETE RESTRICT ON UPDATE CASCADE;
```

Q6. Optimize Nested-Loop Query using JOIN (5 Marks)

Inefficient Query:

```
SELECT Name FROM Employee E
WHERE EXISTS (
    SELECT 1 FROM Works_On W
    WHERE W.EmpID = E.EmpID
    AND EXISTS (
        SELECT 1 FROM Project P
        WHERE P.ProjID = W.ProjID AND P.Budget > 200000
    )
);
```

Optimized with JOIN:

```
SELECT DISTINCT E.Name
FROM Employee E
JOIN Works_On W ON E.EmpID = W.EmpID
JOIN Project P ON W.ProjID = P.ProjID
WHERE P.Budget > 200000;
```

Benefit: Optimizer can use hash/merge joins and indexes instead of nested loops.

Q7. Debug GROUP BY Wrong Aggregation (5 Marks)

Buggy Query:

```
SELECT DeptID, Name, AVG(Salary)
FROM Employee GROUP BY DeptID;
-- Name is neither aggregated nor in GROUP BY
```

Corrected:

```
-- Option A: Remove Name
SELECT DeptID, AVG(Salary) AS AvgSal
FROM Employee GROUP BY DeptID;
```

```
-- Option B: Include Name in GROUP BY
SELECT DeptID, Name, AVG(Salary) AS AvgSal
FROM Employee GROUP BY DeptID, Name;
```

Rule: Every selected non-aggregate column must appear in GROUP BY.

Q8. Fix Logical Error in UPDATE (modifies all rows) (5 Marks)

Buggy Statement:

```
UPDATE Employee SET Salary = Salary * 1.10;
-- Missing WHERE → updates EVERY row
```

Fixed Statement:

```
UPDATE Employee
SET Salary = Salary * 1.10
WHERE DeptID = 'D1' AND Salary < 70000;
```

Always include a precise WHERE clause. Test with SELECT first.

Q9. Optimize Multi-table Query – Subqueries → JOINS + Indexes (5 Marks)

Inefficient Query:

```
SELECT Name FROM Employee
WHERE DeptID IN (SELECT DeptID FROM Department WHERE Location = 'Hyderabad')
AND EmpID IN (SELECT EmpID FROM Works_On WHERE Hours > 30);
```

Optimized Query:

```
SELECT DISTINCT E.Name
FROM Employee E
JOIN Department D ON E.DeptID = D.DeptID
JOIN Works_On W ON E.EmpID = W.EmpID
WHERE D.Location = 'Hyderabad' AND W.Hours > 30;
```

Recommended Indexes:

```
CREATE INDEX idx_emp_deptid ON Employee(DeptID);
CREATE INDEX idx_dept_location ON Department(Location);
CREATE INDEX idx_workson_empid ON Works_On(EmpID);
CREATE INDEX idx_workson_hours ON Works_On(Hours);
```

Q10. Debug Relational Calculus – Incorrect Quantifier (Empty Result) (5 Marks)

Buggy Expression:

```
{ e.Name | Employee(e) ∧ ∀ d (Department(d) ∧ d.DeptName = 'IT'
    ∧ e.DeptID = d.DeptID ∧ e.Salary > 50000) }
```

-- ∀ forces match with ALL departments → empty result

Corrected Expression:

```
{ e.Name | Employee(e) ∧ e.Salary > 50000 ∧  
  ∃ d (Department(d) ∧ d.DeptID = e.DeptID ∧ d.DeptName = 'IT') }
```

Error: Wrong quantifier. Need existential $\exists$ ("there exists"), not universal $\forall$ ("for all").